Operating and Maintenance of the Urban Data Integration Platform

Yixin Xia, Xinyue Feng, Hsi-Jui Chang

September 27, 2026

1 Incremental Processing Strategy

1.1 Incremental Dataset Generation

Generated data is stored under `data/updates/`, while a machine-readable summary is stored in `artifacts/task1_update_manifest.json`. Taxi trips are sampled from valid source records so that the generated release preserves realistic relationships between locations, distance, duration, and fare components. Pickup and drop-off timestamps are shifted beyond the latest pickup time in the original dataset while preserving each trip's duration. Additional records are copied unchanged from the original dataset to exercise duplicate detection. Weather observations cover the 168 hours immediately following the source period. Values are derived from the latest valid observation and adjusted with bounded periodic variation. Air-quality observations are generated for every monitoring configuration over the same 168-hour period. $PM_{2.5}$ measurements are varied within realistic ranges, and AQI is calculated using the US EPA piecewise interpolation method. Table 1 summarises the generated releases. The Taxi release contains 5% new records and approximately

Table 1: Generated incremental datasets

Dataset	Original records	New records	Duplicates	Schema change
Taxi Trips	9,554,778	477,739	95,548	None
Weather	8,784	168	0	Added numeric `humidity` in the range 20–100
Air Quality	8,139,551	177,240	0	Added integer `aqi` in the range 0–500

1% duplicate records relative to the original dataset. The Weather release contains one city-wide observation for each new hour. The Air Quality release contains one observation per monitoring configuration and hour, which results in a larger number of generated rows.

1.2 Incremental Processing Strategy

The incremental pipeline separates update generation from Delta-table modification, allowing each release to be inspected before it is applied.

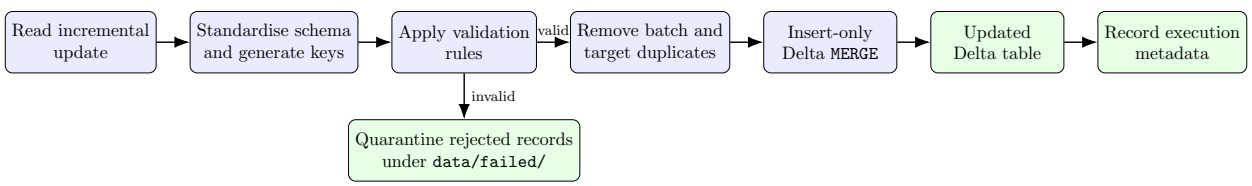


Figure 1: Incremental update workflow

The pipeline retains the Week 1 primary-key definitions: `trip_id` for Taxi Trips, `observation_timestamp` for Weather, and `observation_id` for Air Quality.

Duplicate detection is applied within each incoming batch and against the existing Delta table. Delta `MERGE` inserts records whose primary keys are absent from the target. Existing records remain unchanged, and repeated processing of the same release produces no additional rows.

1.3 Schema Evolution

Weather `humidity` and Air Quality `aqi` are registered as supported additive schema changes. Delta schema evolution adds these columns during the merge. Existing rows remain available and contain `NULL` in the newly added columns, while incoming rows contain the generated values. Existing analytical queries remain compatible when they use the original columns. Deleting columns, changing existing data types, modifying key fields, or introducing unregistered columns requires manual review and a schema-version change.

1.4 Incremental Execution Results

Table 2 reports the first execution of the incremental pipeline. All generated Weather and Air Quality records passed validation. Taxi duplicates were identified and excluded from the processed table. The inserted record counts therefore match the expected release composition.

Table 2: Incremental update execution results

Dataset	Input	Inserted	Duplicates	Time (s)
Taxi Trips	573,287	477,739	95,548	127.3196
Weather	168	168	0	14.7832
Air Quality	177,240	177,240	0	105.8613

2 Maintain Analytical Consistency Strategy

To keep the analytical results consistent after new data is added, we designed the pipeline to update only the parts affected by the new records. First, the pipeline compares the processed taxi table with `integrated_taxi_trips` using `trip_id`. A left anti-join is used to select only taxi trips that have not been integrated before. These new trips are then enriched with taxi-zone, weather, and air-quality information.

Taxi-zone data is relatively small, so it is broadcast during the join. Weather observations are matched using the pickup hour, while air-quality records are aggregated by borough and hour before being joined with taxi trips. If no matching weather or air-quality observation is available, the taxi trip is still kept and the contextual columns remain null. This prevents missing contextual data from removing otherwise valid taxi records.

After enrichment, the new records are inserted into `integrated_taxi_trips` using a Delta merge. Existing trips are not inserted again, which makes the pipeline idempotent. Running the same command multiple times therefore does not create duplicate integrated records.

The same strategy is applied to all six analytical products, while unaffected months remain unchanged. Table 3 summarises the first analytical consistency run.

The new taxi trips affected four months, from March to June 2024. The weather match rate was 100%, while the borough-hour air-quality match rate was approximately 9.95%. The lower air-quality match rate is caused by the more limited spatial and temporal coverage of the monitoring stations. A second execution found no pending trips and therefore skipped all analytical-product writes. This shows that the pipeline is idempotent and avoids unnecessary recomputation.

The new `humidity` and `aqi` columns can be handled automatically because they are additive and nullable. Existing query columns are preserved, and Delta schema evolution allows the new

Table 3: Analytical consistency refresh results

Stage	Records or output rows	Time (s)
Incremental integration	477,739	23.6849
Six analytical products	240,605	28.1925
Complete consistency pipeline	–	57.7710
Idempotent rerun	0	11.1452

columns to be added without rewriting old records. More disruptive changes, such as renamed columns, removed columns, incompatible data types, or changed primary keys, require manual changes because they may break existing queries or product logic.

3 Monitoring Architecture

The monitoring framework converts Task 1 and Task 2 execution results into a unified run model stored under `data/metadata/`. The `ingestion_runs` Delta table records execution performance, record counts, schema versions, validation summaries, and status; `validation_failure_events` supports rule-level failure analysis; and `monitoring_writes` measures monitoring overhead. Additive schema evolution preserves the Week 1 execution history. Idempotent writes based on `(run_id, dataset_name)` also allowed three historical Task 1 results to be backfilled without rerunning the pipeline.

3.1 Monitoring Queries

The monitoring component implements Spark SQL queries for the operational questions required by the platform specification.

3.1.1 Validation failures by dataset.

Validation events are aggregated by dataset and execution. Historical Week 1 records use their rejected-record counts when detailed rule-level events are unavailable. Air Quality currently has

Table 4: Observed validation failures

Dataset	Validation failures	Failing executions
Air Quality	212,184	1
Taxi Trips	137,089	1
Weather	1	1
Taxi Zones	0	0

the highest absolute number of validation failures. The count should be interpreted together with dataset size because Air Quality and Taxi Trips contain substantially more records than Weather and Taxi Zones.

3.1.2 Longest processing time.

The Week 1 full Taxi ingestion is the longest recorded execution at approximately 968.51 seconds. Within the incremental pipeline, Taxi Trips required 127.32 seconds, Air Quality required 105.86 seconds, and Weather required 14.78 seconds. The Taxi update performs primary-key generation, validation, comparison against the existing target, and a Delta merge over the largest incoming batch. These operations account for its longer incremental execution time.

3.1.3 Rejected records per execution.

The execution-level query returns the run identifier, dataset, pipeline type, input count, inserted count, rejected count, and duplicate count. Week 1 recorded 212,184 rejected Air Quality rows, 137,089 rejected Taxi rows, one rejected Weather row, and no rejected Taxi Zone rows. The generated incremental release produced no invalid-value rejections. Its 95,548 intentional Taxi duplicates were reported separately and excluded from the processed table.

3.1.4 Processing-time trend.

A window query uses `LAG` over dataset and pipeline type to compare each execution with its predecessor. Taxi Zones currently has two comparable initial-ingestion executions: processing time increased from approximately 20.14 seconds to 30.91 seconds, an increase of 10.77 seconds. Most dataset and pipeline-type combinations currently contain a single execution. Additional runs will extend the history and make regression and trend analysis more representative.

3.2 Operational Value

Execution time, input volume, rejection rate, duplicate count, validation rule counts, schema version, and execution status provide the most useful operational view. Execution time combined with input volume helps distinguish expected data-volume growth from a performance regression. Rejection rates identify changes in source-data quality. Rule-level failure counts distinguish schema, completeness, value-domain, reference-integrity, and duplicate problems. Schema versions and schema-change descriptions connect operational anomalies to data-model evolution. The run identifier connects monitoring records with quarantined rows. An operator can locate an abnormal execution, identify its dominant validation rules, inspect the associated rejected records, and compare its duration and schema version with earlier runs. This supports debugging without relying on notebook output or manually searching console logs. Additional production monitoring should include executor failures, task retries, shuffle bytes, memory spill, garbage-collection time, input and output bytes, Delta file counts, watermark lag, data freshness, SLA violations, and downstream refresh status. Alerts can be configured for failed executions, unexpected schema changes, elevated rejection rates, and abnormal duration.

4 Extended Validation Framework

The extended validation framework is placed between the Task 1 transformation step and the final Delta merge. This means that incoming data is first transformed into the common format and then validated. Only records returned in the `valid` DataFrame are written to the processed Delta tables. Invalid records are stored separately under `data/failed/<dataset>` and are excluded from the integrated dataset and analytical products.

The framework detects incomplete records, invalid attribute values, missing reference records, duplicate records, and unsupported schema changes. Duplicate detection is performed both within the incoming batch and against primary keys already stored in the target Delta table. Duplicate records are included in `rejected_records`, but they are also counted separately in `duplicate_records`. Therefore, the result always follows:

$$\texttt{input_records} = \texttt{valid_records} + \texttt{rejected_records}. \tag{1}$$

Some parts of the framework are generic and can be reused for every dataset. These include required-field checks, duplicate detection, schema comparison, reference validation, error collection, quarantine writing, and summary generation. Other rules are dataset-specific because different datasets have different meanings and valid ranges. For example, taxi validation checks trip timestamps, distances, fares, passenger counts, and taxi-zone references. Weather validation checks

humidity, precipitation, wind direction, wind speed, and pressure. Air-quality validation checks coordinates, pollutant measurements, and AQI values.

The rules are stored in dataset configuration instead of being written directly into the validation engine. Reusable rule functions such as `numeric_range`, `non_negative`, `not_nan`, and `timestamp_order` are used to define the checks. As a result, a new rule can usually be added by changing the configuration without modifying the core framework. Supporting a new dataset mainly requires defining its primary key, required fields, validation rules, reference rules, and permitted schema changes.

Invalid data does not stop the processing of valid records. Each rejected row contains its validation errors together with the validation run ID, timestamp, dataset name, and schema version. If the framework detects an unsupported schema change, the batch is given the `REJECTED_SCHEMA` status and isolated instead of being merged into the target table.

This design adds some processing overhead because it requires additional joins, counts, and quarantine writes. However, it improves reliability and makes validation failures easier to investigate. The exact validation overhead will be measured separately as part of the platform evaluation.

5 Engineering Decisions

5.1 Task 1

Update generation and Delta-table application are separate operations, enabling release inspection and independent performance measurement. Centralised configuration defines schemas, paths, keys, and schema versions. Stable keys, shared validation rules, and insert-only Delta merges provide reusable and idempotent incremental processing.

5.2 Task 2

Stable `trip_id` values and a left anti-join identify Taxi trips that have not yet been integrated. Recomputing only the affected monthly slices preserves derived metrics while avoiding a full historical rebuild. Broadcast joins and time-range filtering further reduce unnecessary shuffles and scans.

5.3 Task 3

A unified run model supports ingestion, incremental updates, and analytical refreshes. Additive schema evolution retains Week 1 monitoring history, while rule-level events support Spark SQL analysis. Idempotent monitoring writes also allow historical results to be backfilled safely.

5.4 Task 4

Validation rules are stored in dataset-specific configuration instead of the core engine. The engine handles required fields, duplicates, schema changes, reference checks, quarantine writes, and summary generation in a reusable way. Validation runs before the Delta merge so that only valid records enter the processed tables.

6 Trade Offs

6.1 Task 1

Exact sampling and target-key comparison introduce additional Spark scans. Template-based generation preserves realistic source characteristics but cannot represent every future data pattern. Insert-only merges handle new and duplicate records safely, while corrections using existing keys require a separate update policy. New schema fields contain `NULL` for historical rows until a backfill is performed.

6.2 Task 2

Recomputing complete monthly slices is more expensive than updating simple counts, but it keeps averages, distributions, and other derived values consistent. The current dependency check is driven by new Taxi trips, so corrections to historical Weather or Air Quality records require an explicit refresh.

6.3 Task 3

Rule-level monitoring improves diagnosis while adding storage and Delta-write overhead. Local monitoring writes required approximately 16–34 seconds due to Spark startup and small transactions; batching and persistent Spark sessions can reduce this cost. Non-blocking monitoring protects pipeline availability, although failed monitoring writes may leave gaps in operational history.

6.4 Task 4

Duplicate and reference checks require additional joins and counts, while quarantined rows introduce extra Delta storage. Rejecting an incompatible schema protects the target table but may isolate an entire batch even when some rows are otherwise valid. Dataset-specific thresholds also need review when source definitions change.